UDP Log Viewer

Cross-platform UDP log viewer for ESP32 and other embedded systems, built with Python and PyQt5.

Why using this tool?

Debugging embedded devices over UDP is painful. This tool makes it simple and visual. Not only viewing data, visualizing data as well.

Documentation

Complete English documentation:

- docs/DOCUMENTATION_en.md
- docs/USER_GUIDE_en.md
- docs/RELEASE_0.16.2.md

Complete German documentation:

- docs/DOKUMENTATION_de.md

HowTo use UDP on EPS32 Demo project

If you want to play around with an ESP32 & logging, please visit this demo repository https://github.com/mrRobot62/esp-udp-logging.git

Windows-User

please use setup.exe to install this viewer

ToDo for MAC users

after installation this application is not signed and can't be started directly. You have to set this command: `xattr -dr com.apple.quarantine /Applications/UDPLogViewer.app`

Screenshots

Screenshot assets are stored under assets/screenshots.

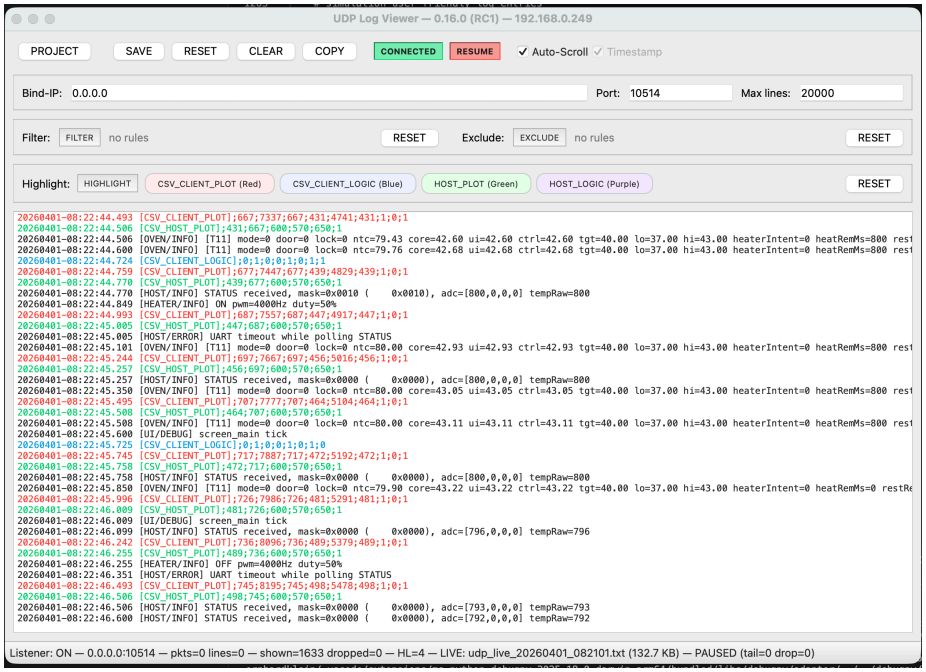


Figure 1: UDP Log Viewer main screen with active connection, reset workflow, and rule chips

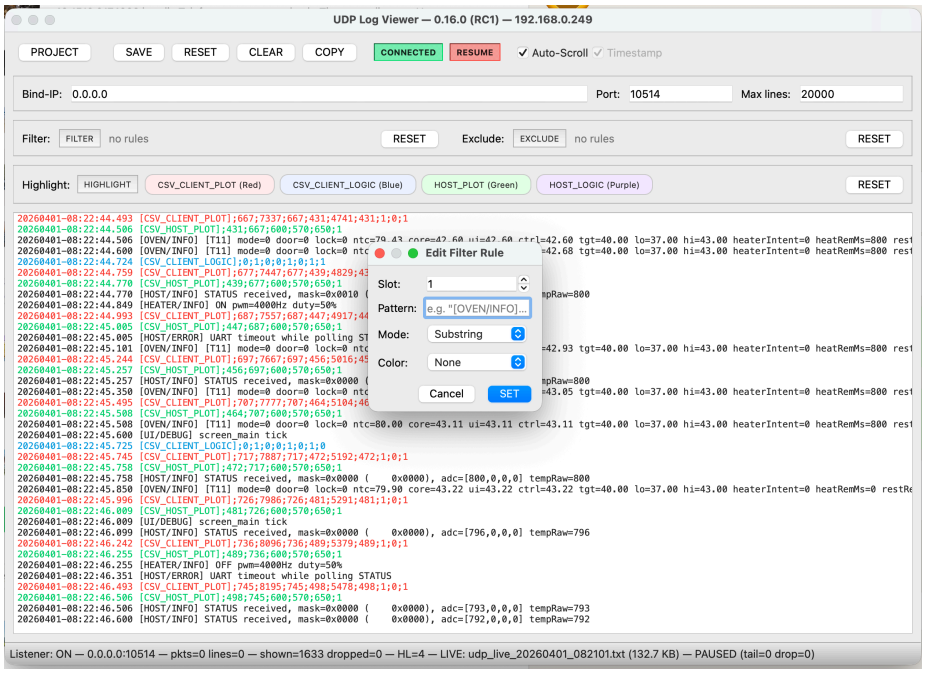


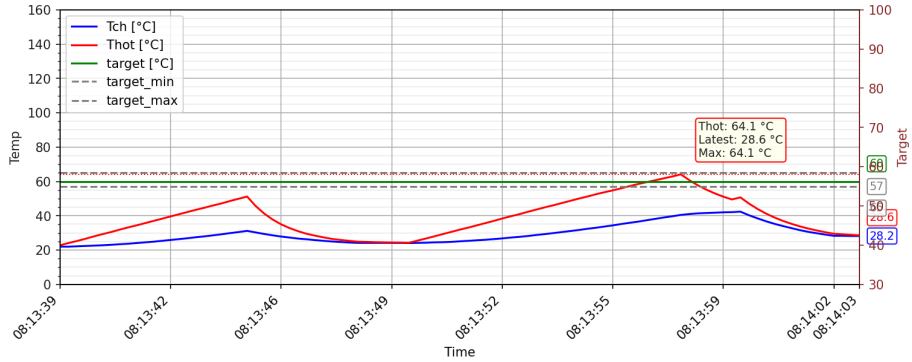
Figure 2: UDP Log Viewer filter-rule dialog on top of the main window

Main Screen

Rule Configuration

Project Dialog

Plot Visualizer Tooltip



Timebased visualizing

Measuring time

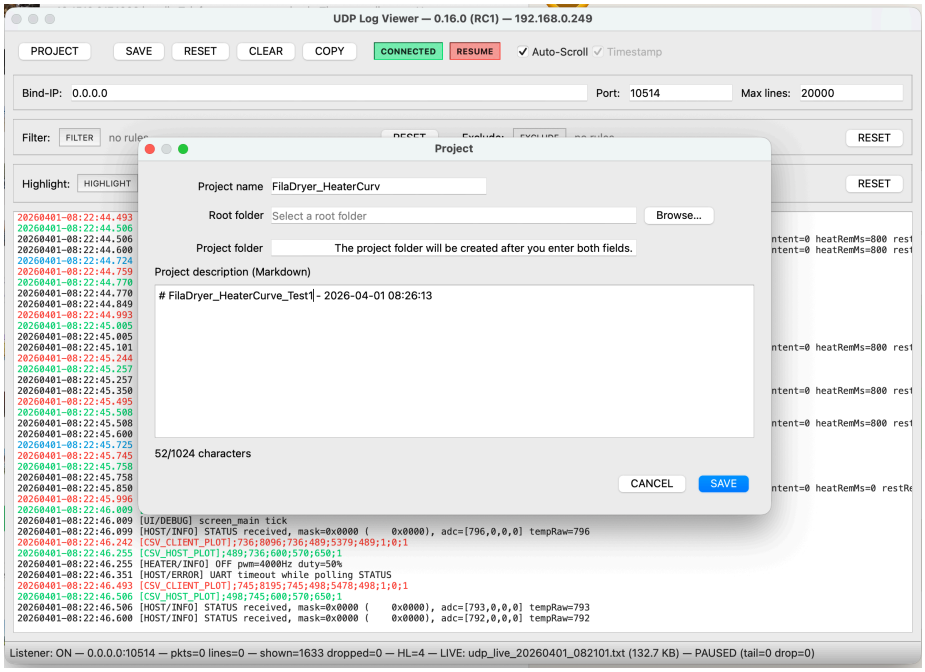


Figure 3: UDP Log Viewer project dialog with Markdown project description

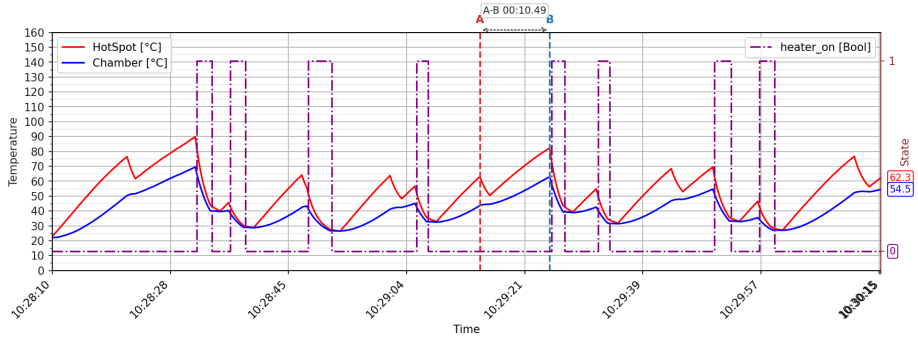
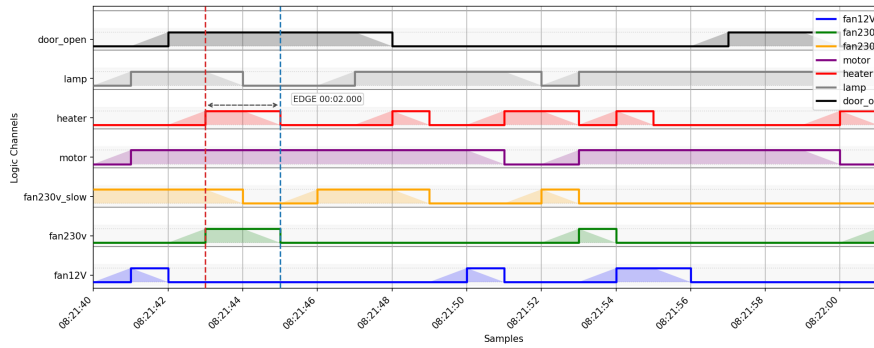
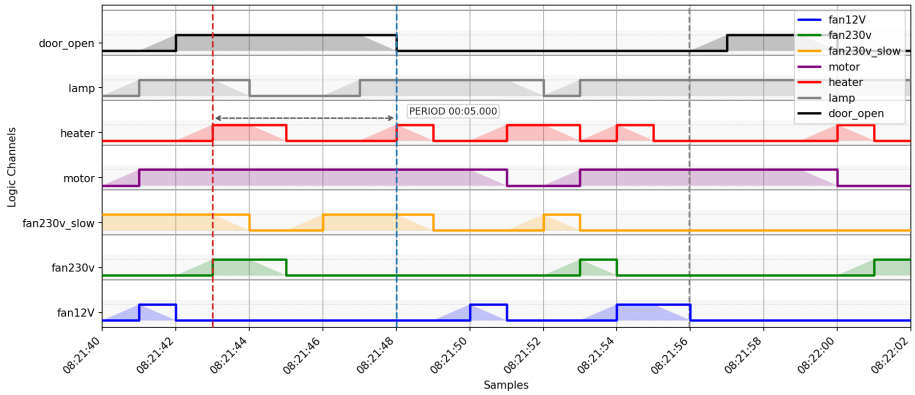


Figure 4: Measure timing

Logic Measurement



Measuring HIGH/LOW phase



Measuring signal period

Current Scope

The current codebase includes:

- real-time UDP log reception
- filter, exclude, and highlight rules
- live session logging
- main-window `RESET` support to start a fresh log phase inside the same app session
- runtime project contexts for grouping logs and screenshots per test session
- project descriptions stored as `README_<projectname>.md` inside the project folder
- replay of saved log files
- built-in simulation for text, temperature, and logic traffic
- CSV-based data visualization
- logic-channel visualization
- edge and period measurement directly inside the logic graph
- runtime `Legend` toggle in plot and logic graph windows
- keyboard-driven screenshot/save shortcuts and explicit `TAB` navigation in

the main and graph windows
- macOS and Windows packaging scripts

Run From Source

```
python -m venv venv
source venv/bin/activate
pip install -e .[dev]
udp-log-viewer
```

Developer Bootstrap

```
./scripts/bootstrap_macos_linux.sh
```

You like it?

If you like this project, consider giving it a star!